Rapport du projet de fin d'année

Filière: INFO4

Micro-Service et Micro-Frontend pour SPLAT

Encadré par :

Mr DONSEZ Didier

Réalisé par :

ELOUAZZANI Soufian

BOUTI Chaimae

Université Grenoble Alpes

Polytech Grenoble - INP

Promotion : 2026/2027

Grenoble, mars 2026

Remerciements

Nous avons éprouvé une grande satisfaction en exprimant notre gratitude à tous ceux qui ont contribué au succès de ce projet.

Nous souhaitons adresser nos sincères remerciements à notre encadrant, **M. Donsez Didier**, pour son encadrement bienveillant, sa disponibilité et les précieux conseils qu'il nous a prodigués tout au long de ce projet.

Nous remercions également **M. Letellier François** pour son aide précieuse dans la gestion et l'organisation du projet, ainsi que pour son soutien constant qui nous a permis de maintenir le cap tout au long de cette expérience.

Nos remerciements vont aussi à **M. Palix Nicolas** et **M. Lechevalier Pierre** pour leurs conseils avisés et leurs remarques constructives, qui ont grandement contribué à améliorer la qualité de notre travail.

Enfin, nous remercions toutes les personnes ayant contribué, de près ou de loin, à l'élaboration de ce rapport.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un projet de 4ème année à **Polytech Informatique**.

Nous avons travaillé sur l'évolution de l'application **Meshtastic MQTT Explorer**, un outil de supervision en temps réel des réseaux mesh LoRa. Nous avons intégré un service de **calcul de couverture radio** basé sur le simulateur SPLAT!, ainsi qu'une architecture **Micro Frontend** isolant la carte de couverture dans une application Vue.js indépendante. Cette approche minimise les modifications du code existant tout en facilitant les évolutions futures.

Lien du projet : https://github.com/soufian-elouazzani/meshnetwork

Table des matières

Table des matières **5**

1 Introduction Générale **6**

2 Contexte Général du Projet **7**

2.1 Introduction 7

2.2 Architecture du Réseau Meshtastic 7

2.2.1 Présentation du Protocole Meshtastic 7

2.2.2 Composants et Rôles du Réseau 8

2.3 Analyse du Système Existant : Meshtastic MQTT Explorer 8

2.3.1 Présentation de l'Architecture 8

2.3.2 Flux de données 9

2.3.3 Composants principaux 10

2.3.4 Communication entre les services 10

2.3.5 Limites du système actuel 10

2.4 Conclusion 11

3 Gestion De Projet **12**

3.1 Introduction 12

3.2 Méthode de gestion 12

3.3 Planning et calendrier 13

3.4 Conclusion 14

4 Architecture Microservices et Approche Open Source **15**

4.1 Introduction 15

4.2 Pourquoi les Microservices ? 15

4.2.1 Conteneurisation avec Docker 16

4.3 Choix Technologiques . . . 16
4.3.1 SPLAT! pour le calcul de propagation radio . . . 16
4.3.2 PostgreSQL pour la persistance . . . 16
4.3.3 Architecture de communication . . . 17
4.3.4 Vue.js pour le Microfrontend . . . 17
4.4 Impact sur le Projet Open Source . . . 18
4.4.1 Contributions potentielles . . . 18
4.4.2 Réutilisabilité des services . . . 18
4.5 Critères de Qualité Visés . . . 19
4.5.1 Performance . . . 19
4.5.2 Maintenabilité . . . 19
4.5.3 Portabilité . . . 19
4.5.4 Sécurité . . . 19
4.6 Points Critiques de l'Application . . . 20
4.6.1 Temps de calcul SPLAT! . . . 20
4.6.2 Intégration avec le Frontend .NET Blazor . . . 20
4.6.3 Persistance des données de démonstration . . . 20
4.6.4 Compatibilité des versions PostgreSQL . . . 20
4.7 Conclusion . . . 21

5 Services de Couverture, Redondance et Microfrontend 22
5.1 Introduction . . . 22
5.2 Service de Couverture Radio . . . 22
5.2.1 Architecture du service . . . 22
5.2.2 Principe de fonctionnement . . . 23
5.2.3 Composants clés . . . 23
5.2.4 Code clé – Boucle principale . . . 24
5.2.5 Code clé – Appel à SPLAT! . . . 24
5.2.6 Stockage des résultats . . . 25
5.2.7 Visualisation des logs . . . 25
5.3 Service de Redondance . . . 26
5.3.1 Architecture du service . . . 26
5.3.2 Objectif et principe . . . 26

5.3.3 Algorithme de calcul des chevauchements . . . 27
5.3.4 Niveaux de redondance et couleurs . . . 27
5.3.5 Stockage des résultats . . . 28
5.3.6 Composants du service . . . 28
5.3.7 Dockerfile du service . . . 29
5.3.8 Visualisation des logs . . . 29
5.4 Intégration d'une Architecture Micro Frontend . . . 30
5.4.1 Contexte et Motivation . . . 30
5.4.2 Concept du Micro Frontend . . . 30
5.4.3 Choix Technologiques . . . 31
5.4.3.1 Vue.js 3 pour le Micro Frontend . . . 31
5.4.3.2 Intégration via iframe . . . 31
5.4.4 Architecture de Déploiement . . . 31
5.4.5 Résultats . . . 32
5.5 Conclusion . . . 34

6 Conclusion Générale **35**

Bibliographie **36**

Chapitre 1

Introduction Générale

Ce projet s'inscrit dans le cadre de l'évolution de **Meshtastic MQTT Explorer**, un outil de supervision en temps réel des **réseaux maillés LoRa**. L'objectif principal était d'enrichir cette solution existante avec des fonctionnalités avancées d'analyse de signal, tout en relevant un défi architectural majeur : intégrer ces nouveautés sans modifier en profondeur le code source original en .NET .

Pour répondre à ce besoin, nous avons adopté une architecture moderne basée sur les **microservices** et **le Micro Frontend**. Ce travail s'est articulé autour de trois axes majeurs :Le développement d'un **service de couverture radio** utilisant **le simulateur SPLAT**! pour prédire la portée des nœuds en fonction du relief.La mise en place d'**un service de redondance** pour identifier les zones de chevauchement et évaluer la résilience du réseau.L'intégration d'une interface indépendante en Vue.js, encapsulée via une iframe, permettant une visualisation cartographique fluide et isolée du reste de l'application. Ce projet s'inscrit dans le cadre de l'évolution de **Meshtastic MQTT Explorer**, un outil de supervision en temps réel des **réseaux maillés LoRa**. L'objectif principal était d'enrichir cette solution existante avec des fonctionnalités avancées d'analyse de signal, tout en relevant un défi architectural majeur : intégrer ces nouveautés sans modifier en profondeur le code source original en .NET.

Chapitre 2

Contexte Général du Projet

2.1 Introduction

Ce chapitre pose le cadre contextuel et technique indispensable à la compréhension des enjeux du projet. Dans un premier temps, nous détaillons l'architecture du réseau Meshtastic, un protocole de communication maillé décentralisé reposant sur la technologie radio LoRa. Nous y présentons les différents composants de l'écosystème — nœuds, passerelles et courtiers MQTT — ainsi que leurs rôles respectifs dans la transmission des données.Par la suite, nous procédons à une analyse critique du système existant, le Meshtastic MQTT Explorer. Cette étude nous permet de décortiquer son architecture microservices actuelle et son pipeline de données afin d'identifier précisément les limitations techniques qui ont motivé nos travaux. Cette étape est cruciale pour justifier l'intégration des nouveaux services de couverture et de redondance que nous avons développés.

2.2 Architecture du Réseau Meshtastic

2.2.1 Présentation du Protocole Meshtastic

Meshtastic est un protocole de communication maillé open source qui fonctionne sur la technologie radio LoRa. Il permet une communication décentralisée entre les appareils sans dépendre des réseaux cellulaires ou de l'infrastructure Internet. Le protocole utilise la modulation LoRa pour atteindre une portée étendue tout en maintenant une faible consommation d'énergie, ce qui le rend idéal pour les activités de plein air, les communications d'urgence et les applications IoT.

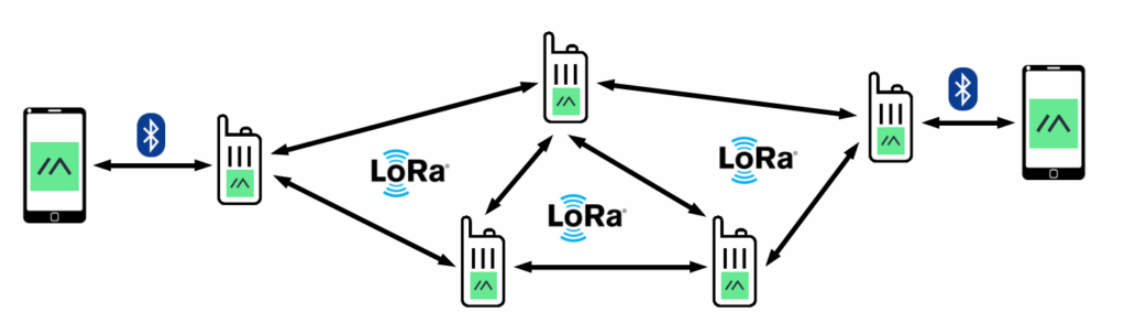

Figure 2.1: Topologie du réseau maillé Meshtastic

2.2.2 Composants et Rôles du Réseau

L'écosystème Meshtastic comprend plusieurs composants clés qui fonctionnent ensemble pour former un réseau maillé fonctionnel :

- **Nœuds (Nodes)** : Appareils Meshtastic individuels (RAK, T-Beam, etc.) qui forment le réseau maillé. Chaque nœud peut relayer des messages, partager des données de position et communiquer avec d'autres nœuds à portée.
- **Nœuds Passerelle (Gateway Nodes)** : Nœuds spécialisés avec connectivité Internet (WiFi ou Ethernet) qui font le pont entre le maillage LoRa et les courtiers MQTT. Ces passerelles permettent une connectivité mondiale en transférant le trafic maillé local vers des services basés sur Internet.
- **Courtiers MQTT (MQTT Brokers)** : Centres de messages centraux qui reçoivent et distribuent les données Meshtastic des nœuds passerelle. Le courtier public officiel (mqtt.meshtastic.org) et les courtiers privés (comme Mosquitto) servent de points d'agrégation pour les données maillées.

2.3 Analyse du Système Existant : Meshtastic MQTT Explorer

2.3.1 Présentation de l'Architecture

Le Meshtastic MQTT Explorer, développé par Valentintintin, est une solution de surveillance complète construite sur la technologie .NET. Il fournit une visualisation en temps réel et une analyse historique de l'activité du réseau Meshtastic via une interface web. Le système suit un modèle d'architecture microservices avec les composants suivants :

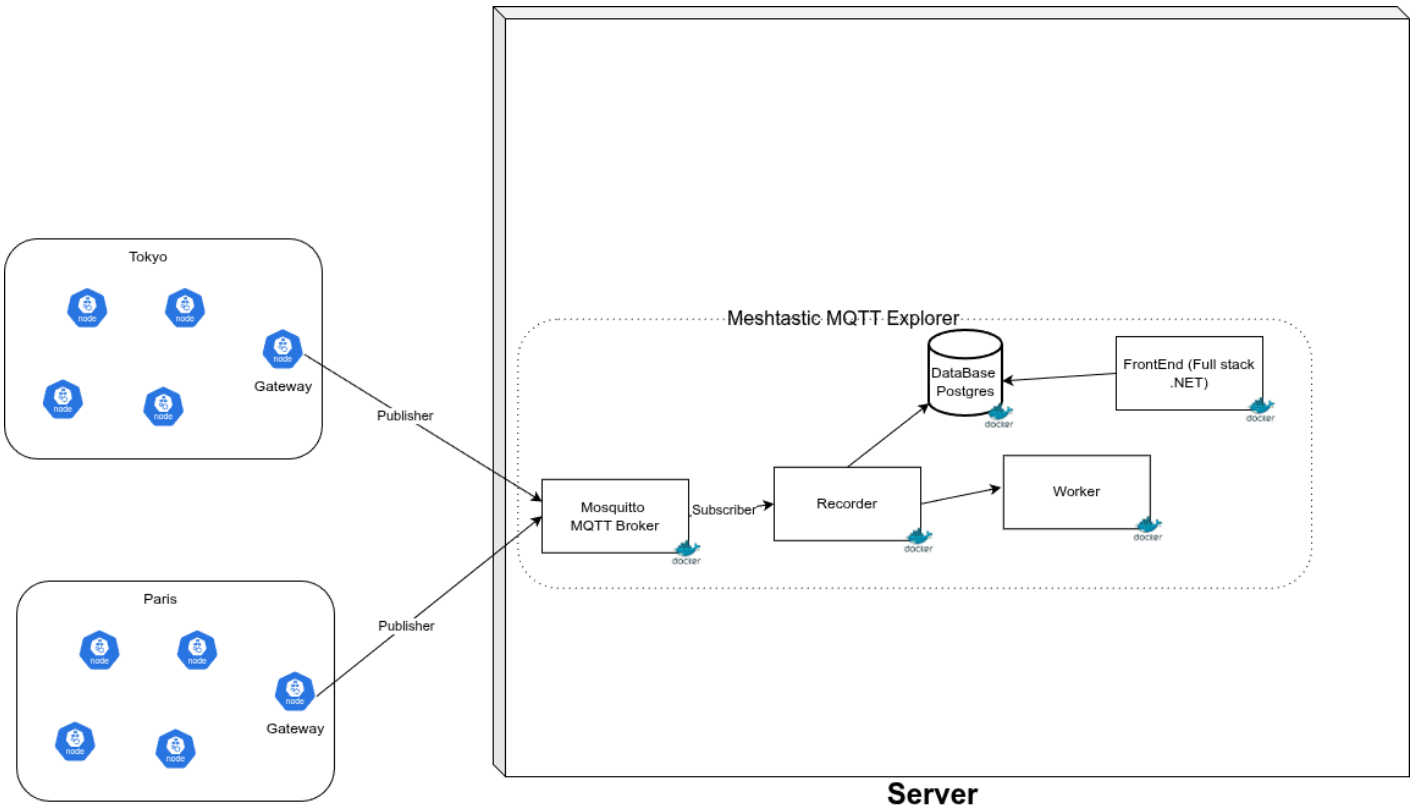

Figure 2.2: Architecture existante du Meshtastic MQTT Explorer

2.3.2 Flux de données

Le fonctionnement du système repose sur un pipeline de données en plusieurs étapes :

1. **Collecte des données** : Les nœuds Meshtastic sur le terrain communiquent via le réseau LoRa. Un nœud passerelle (*Gateway*) connecté à Internet relaie les messages vers un courtier MQTT.

2. **Transmission MQTT** : Les données sont publiées sur un courtier Mosquitto (port 1883) qui sert de concentrateur central.

3. **Ingestion et persistance** : Le service `Recorder` se connecte au courtier MQTT, décode les paquets Meshtastic et les enregistre dans la base de données PostgreSQL.

4. **Visualisation** : Le service `Front` est une application Blazor full-stack qui :

   - Interroge directement la base de données via ses contrôleurs API
   - Génère une interface cartographique avec Leaflet
   - Se connecte également au courtier MQTT pour les mises à jour en temps réel

5. **Tâches de fond** : Le service `Worker` exécute des tâches périodiques (nettoyage des données, calculs de statistiques).

2.3.3 Composants principaux

Le tableau 2.1 présente les cinq services composant l'architecture existante ainsi que leur rôle respectif.

Service	**Rôle**
`Mosquitto`	Courtier MQTT centralisant tous les messages des passerelles
`Recorder`	Écoute le MQTT, décode les paquets, écrit en base de données (port 81)
`Database`	PostgreSQL stockant les nœuds, positions, messages et historiques
`Front`	Application Blazor full-stack servant l'interface web (port 80)
`Worker`	Exécute les tâches de fond (purge des données, statistiques)

Table 2.1: Description des services existants

2.3.4 Communication entre les services

- **Données historiques** : Le Front interroge directement la base de données PostgreSQL via Entity Framework Core.
- **Données temps réel** : Le Front établit une connexion WebSocket vers le courtier MQTT pour recevoir les mises à jour en direct.
- **API REST** : Le service Recorder expose des endpoints sur le port 81 utilisés par le Front.

2.3.5 Limites du système actuel

Bien que fonctionnel, le système présente certaines limitations :

- **Absence de prédiction de couverture** : Aucune analyse de la portée radio des nœuds

- **Manque d'analyse de redondance** : Impossible d'identifier les zones couvertes par plusieurs nœuds
- **Intégration difficile** : Toute nouvelle fonctionnalité nécessite de modifier le code source .NET

Ces limitations ont motivé l'ajout des services de couverture et de redondance présentés dans le chapitre suivant.

2.4 Conclusion

L'analyse du système existant a révélé un outil de supervision performant mais dont l'évolution est freinée par une structure monolithique pour sa partie interface. L'absence de prédiction de portée radio constitue une lacune importante pour l'optimisation d'un réseau maillé LoRa.Ces constats nous ont permis de définir des objectifs clairs d'amélioration technique. Avant de détailler la mise en œuvre de notre solution, il est essentiel d'aborder la méthodologie adoptée pour structurer ce travail. Le chapitre suivant sera donc consacré à la gestion du projet et à la planification, où nous présenterons l'organisation de nos ressources et le calendrier des étapes clés ayant mené à la réalisation des services de couverture et de redondance.

Chapitre 3

Gestion De Projet

3.1 Introduction

Après avoir défini les enjeux techniques et le contexte de l'application Meshtastic MQTT Explorer , ce chapitre détaille la structure organisationnelle et méthodologique adoptée pour mener à bien ce projet. Compte tenu de la nature collaborative du travail en binôme, il était essentiel de définir une démarche rigoureuse pour synchroniser nos développements et respecter les échéances imposées. Nous présentons ici les modalités de gestion, la répartition des rôles entre les membres de l'équipe, ainsi que le calendrier de réalisation qui a rythmé nos dix semaines de travail.

3.2 Méthode de gestion

Le projet a adopté une démarche itérative et collaborative. Les principales modalités de travail sont les suivantes :

- **Réunions binôme** : Ayant pour objectif de synchroniser les avancées, d'intégrer les développements de chacun et de planifier les prochaines tâches.
- **Encadrement hebdomadaire** : des réunions hebdomadaires avec M. Donsez Didier assuraient un suivi régulier du projet, permettant de valider les choix techniques et de réorienter le travail si nécessaire.
- **Présentation intermédiaire** : une présentation d'avancement a été réalisée à mi-parcours devant l'encadrant, permettant d'évaluer la progression par rapport aux objectifs initiaux et d'ajuster la feuille de route pour la seconde moitié du projet.

Cette organisation légère, adaptée à une équipe de deux personnes, a permis de maintenir une communication fluide et une progression continue tout au long du projet.

Rôles et répartition des tâches

La répartition des tâches a été définie en début de projet en fonction des compétences et des affinités de chacun, comme le résume le tableau 3.1.

Membre	**Tâches réalisées**
ELOUAZZANI Soufian	• Développement du service de couverture radio (SPLAT!) • Développement du service de redondance • Conteneurisation des deux services avec Docker
BOUTI Chaimae	• Développement du Micro Frontend Vue.js • Intégration via iframe dans le shell Blazor • Communication avec le backend via les endpoints REST

Table 3.1: Répartition des tâches entre les membres de l'équipe

Nous avons collaboré conjointement sur la conception de l'architecture globale, la rédaction du rapport et les tests d'intégration de l'ensemble des services.

3.3 Planning et calendrier

La figure suivante illustre ce planning sous forme de diagramme de GANTT. On observe que le projet s'est déroulé sur dix semaines, de janvier à fin mars 2026. Les deux premières semaines ont été consacrées à la prise en main du projet existant, suivies d'une phase de développement des services backend (couverture, redondance) de fin janvier à début mars. Le développement du frontend et la

rédaction du rapport ont débuté en parallèle à partir de fin février, permettant d'optimiser le temps restant avant la livraison finale.

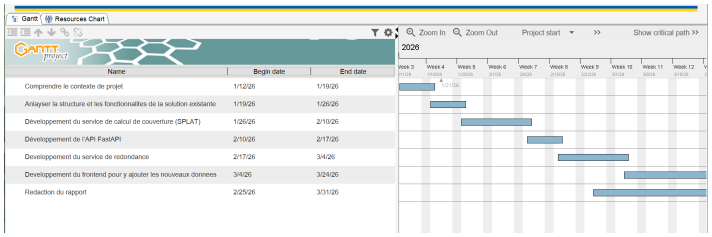

Figure 3.1: Plannification en Diagramme de GANTT

Le projet s'est appuyé sur deux séances de travail par semaine d'environ 3 heures chacune, soit un volume total approximatif de 60 heures de travail par étudiant sur l'ensemble du projet. Deux jalons importants ont rythmé l'avancement : une présentation intermédiaire permettant de valider l'architecture et les premiers résultats, et une présentation finale le 30 mars 2026 devant l'encadrant pour la soutenance du projet.

3.4 Conclusion

La gestion itérative du projet a permis de transformer les objectifs initiaux en livrables concrets, tout en assurant une montée en compétence équilibrée sur les services backend et frontend. Le respect du diagramme de GANTT a été rendu possible par une communication fluide et une autonomie dans la réalisation des tâches spécifiques à chaque membre. Cette organisation solide nous a permis d'aborder sereinement la phase de conception technique. Le chapitre suivant détaille ainsi l'architecture microservices retenue, choix central qui garantit l'indépendance et la flexibilité de notre solution.

Chapitre 4

Architecture Microservices et Approche Open Source

4.1 Introduction

La phase de planification ayant permis de définir les objectifs et les jalons du projet, ce chapitre se concentre sur les choix fondamentaux d'architecture et les technologies retenues pour leur mise en œuvre. L'un des principaux défis de ce travail consistait à enrichir une application existante, développée en .NET Blazor, sans en altérer le code source original tout en garantissant une intégration fluide des nouvelles fonctionnalités.

4.2 Pourquoi les Microservices ?

Le choix d'une architecture microservices pour nos développements a été motivé par plusieurs objectifs clés :

- **Indépendance** : Chaque service peut être développé, testé et déployé indépendamment des autres.
- **Non-régression** : L'ajout de nouvelles fonctionnalités ne nécessite aucune modification du code source existant.
- **Technologie flexible** : Chaque service peut utiliser le langage le plus adapté (Python pour SPLAT!, Vue.js pour le frontend).
- **Scalabilité** : Les services peuvent être dupliqués pour supporter des charges plus importantes.

Cette approche contraste avec l'architecture monolithique du projet original, où toute modification implique une recompilation complète de l'application .NET.

4.2.1 Conteneurisation avec Docker

L'ensemble des services est conteneurisé avec Docker, permettant une isolation complète et une portabilité maximale. Chaque service possède son propre `Dockerfile` et est exécuté dans son propre conteneur, ce qui facilite :

- Le déploiement sur n'importe quelle machine
- La gestion des dépendances (SPLAT!, Python, nginx)
- La reproductibilité des environnements

4.3 Choix Technologiques

4.3.1 SPLAT! pour le calcul de propagation radio

Le cœur du service de couverture repose sur **SPLAT!** (Signal Propagation, Loss, And Terrain), un logiciel de simulation radio développé par John A. Magliacane, KD2BD. Il utilise les données d'élévation SRTM (Shuttle Radar Topography Mission) de la NASA pour calculer la propagation des signaux en tenant compte du relief.

4.3.2 PostgreSQL pour la persistance

La base de données PostgreSQL assure le stockage des données de couverture et de redondance. Les nouvelles tables ajoutées sont présentées dans le tableau 4.1.

Table	**Rôle**
`CoverageMaps`	Stocke les polygones GeoJSON des zones de couverture
`Redundancy`	Contient les métriques de redondance par nœud
`RedundancyMap`	Agrège les zones de redondance par niveau (1 à 5)

Table 4.1: Nouvelles tables PostgreSQL

4.3.3 Architecture de communication

Notre architecture repose sur un principe de communication indirecte entre les services :

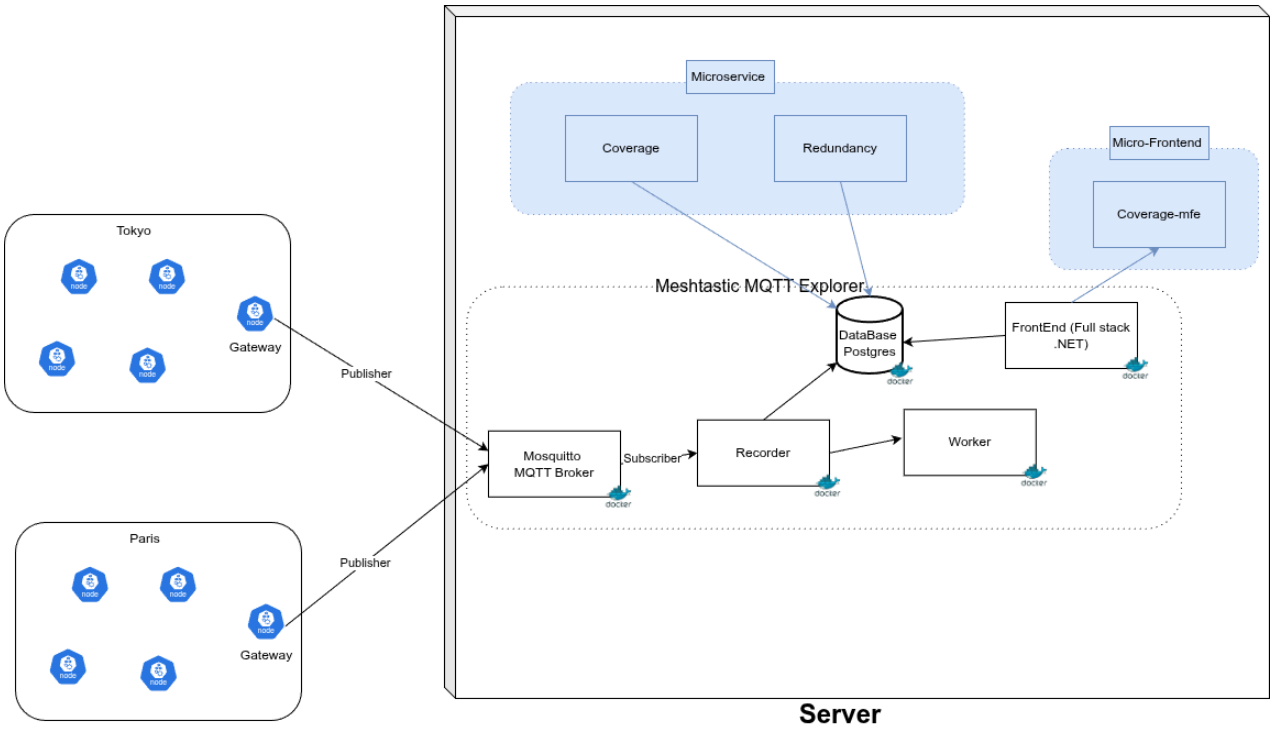

Figure 4.1: Communication entre les services via la base de données

Fonctionnement :

- Le service de couverture (`coverage`) lit les nœuds depuis la table `Nodes` et écrit ses résultats dans `CoverageMaps`.
- Le service de redondance (`redundancy`) lit les données de `CoverageMaps` et `Nodes` pour calculer les chevauchements, puis écrit dans `Redundancy` et `RedundancyMap`.
- Le microfrontend (`coverage-mfe`) communique directement avec l'API du frontend .NET (port 81) pour récupérer les données de couverture et de redondance.
- Le frontend .NET existant (`front`) lit toutes les tables via ses contrôleurs et sert les données à l'interface principale.

4.3.4 Vue.js pour le Microfrontend

Le microfrontend d'affichage de la couverture est développé avec **Vue.js**, un framework JavaScript progressif. Il est servi par un conteneur nginx et communique avec l'API du frontend .NET pour récupérer les données GeoJSON.

```
// coverage-mfe/src/App.vue
const API_BASE = ''  // Utilise le m me serveur que le frontend

async function fetchCoverage() {
  // R cup ration des cartes de couverture via l'API du frontend .NET
  const res = await fetch(`${API_BASE}/api/coverage/all`)
  const data = await res.json()
  // Ajout des calques GeoJSON    la carte Leaflet
  data.forEach(item => {
    L.geoJSON(item.geoJson).addTo(map)
  })
}
```

Listing 4.1: Appel API depuis le microfrontend

4.4 Impact sur le Projet Open Source

4.4.1 Contributions potentielles

Notre travail s'inscrit dans une logique de contribution à l'open source. Les services développés pourraient être proposés à l'auteur original du projet `Meshtastic MQTT Explorer`. Plusieurs atouts plaident en faveur de cette intégration :

- **Zéro modification du code existant** : L'architecture microservices permet d'ajouter des fonctionnalités sans risquer de régressions.
- **Déploiement simple** : L'ajout des services se fait via quelques lignes dans le fichier `docker-compose.yaml`
- **Documentation complète** : Les services sont documentés et prêts à l'emploi.
- **Images Docker publiques** : Les images sont disponibles sur Docker Hub pour une installation facilitée.

4.4.2 Réutilisabilité des services

Les services de couverture et de redondance ne sont pas spécifiques à Meshtastic. Ils pourraient être adaptés pour :

- Tout réseau de communication nécessitant une analyse de couverture radio

- Des projets de planification de réseaux IoT
- Des études de propagation de signaux

4.5 Critères de Qualité Visés

4.5.1 Performance

- Calculs SPLAT! exécutés en arrière-plan toutes les 24h (intervalle configurable)
- Mise en cache des données de terrain pour accélérer les calculs
- Requêtes SQL optimisées avec indexation

4.5.2 Maintenabilité

- Code Python documenté avec docstrings
- Logs structurés pour le débogage
- Architecture modulaire facilitant l'ajout de nouvelles fonctionnalités

4.5.3 Portabilité

- Conteneurisation Docker complète
- Images publiques sur Docker Hub
- Configuration par variables d'environnement
- Aucune dépendance système externe requise

4.5.4 Sécurité

- Variables sensibles (mots de passe) exclusivement via variables d'environnement
- Isolation des conteneurs (réseaux Docker dédiés)
- Aucune donnée stockée en clair dans le code source

4.6 Points Critiques de l'Application

4.6.1 Temps de calcul SPLAT!

Le principal défi technique réside dans la durée des calculs de propagation. Une simulation pour un seul nœud peut prendre entre 10 et 30 secondes, selon la zone géographique et la résolution des données de terrain.

Mitigation :

- Calculs asynchrones en arrière-plan
- Mise en cache des résultats
- Intervalle de calcul configurable (par défaut 24h)
- Possibilité de déclencher manuellement un calcul

4.6.2 Intégration avec le Frontend .NET Blazor

Le frontend existant étant une application Blazor compilée, son code source n'est pas directement modifiable. Pour afficher nos données, nous avons dû :

- Proposer un microfrontend indépendant (Vue.js) pour la visualisation
- Utiliser l'API existante du frontend .NET (port 81) pour exposer nos données
- Ajouter des contrôleurs dans le projet .NET pour servir les données de couverture et de redondance

4.6.3 Persistance des données de démonstration

Lorsqu'un utilisateur clone le projet, la base de données est vide. Pour garantir une expérience utilisateur fluide, un script d'initialisation charge automatiquement des données de démonstration si aucune donnée n'est présente.

4.6.4 Compatibilité des versions PostgreSQL

Les migrations de base de données doivent être gérées avec soin pour éviter les conflits entre les schémas existants et les nouvelles tables. L'utilisation de `ON CONFLICT DO NOTHING` permet d'éviter les erreurs lors des insertions répétées.

4.7 Conclusion

Ce chapitre a présenté les choix architecturaux et technologiques guidant notre projet. L'approche microservices, combinée à la conteneurisation Docker, garantit une intégration non invasive avec le système existant tout en offrant une flexibilité maximale. Les critères de qualité définis (performance, maintenabilité, portabilité, sécurité) servent de cadre pour l'évaluation des livrables. Les points critiques identifiés feront l'objet d'une attention particulière lors du développement et des tests.

Chapitre 5

Implémentation des Services de Couverture, Redondance et Microfrontend

5.1 Introduction

Ce chapitre détaille l'implémentation technique des trois composants principaux développés pour enrichir l'application Meshtastic MQTT Explorer :

- **Service de couverture radio** : calcul automatique des zones de couverture à l'aide de SPLAT!.
- **Service de redondance** : analyse des chevauchements pour identifier les zones redondantes.
- **Microfrontend** : interface de visualisation dédiée, développée avec Vue.js et Leaflet.

Chaque service est conteneurisé, communique via la base de données PostgreSQL et s'intègre sans modification du code source existant.

5.2 Service de Couverture Radio

5.2.1 Architecture du service

Comme illustré sur la figure ci-dessous, le service de couverture fonctionne selon une boucle continue qui orchestre l'ensemble du processus de calcul.

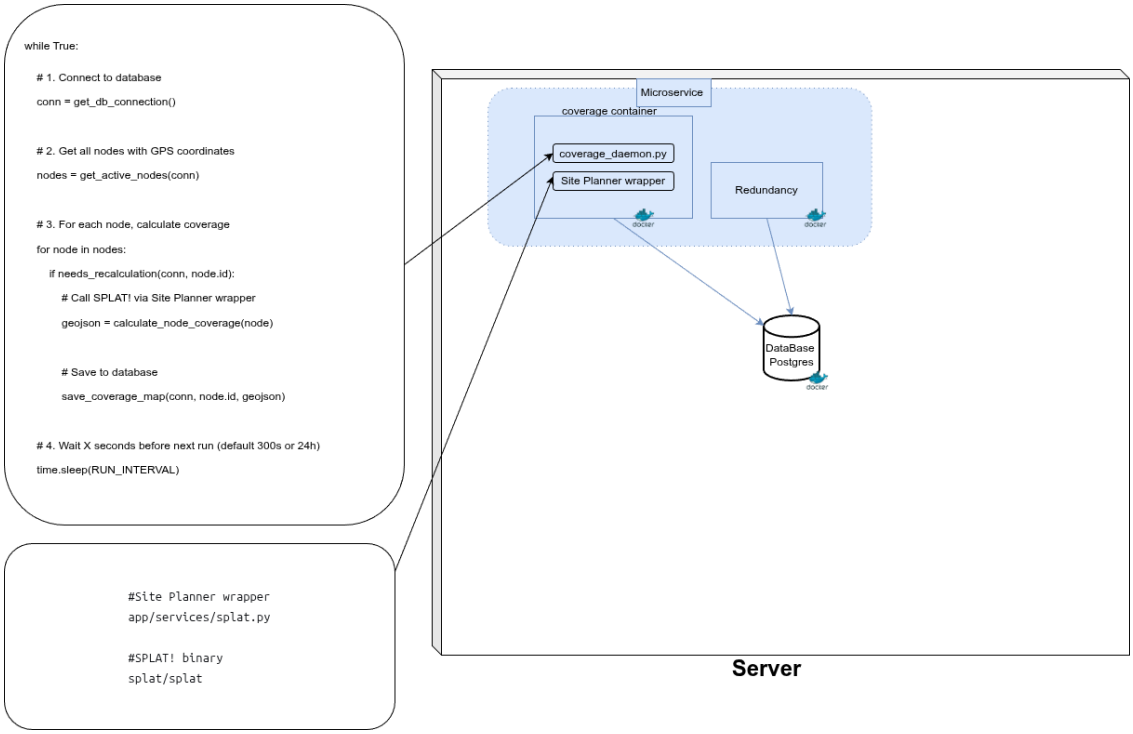

Figure 5.1: Architecture et flux d'exécution du service de couverture

5.2.2 Principe de fonctionnement

Le service de couverture s'exécute en arrière-plan de manière périodique (intervalle configurable). Son fonctionnement suit un cycle en quatre étapes :

1. **Connexion à la base de données** : Le service se connecte à PostgreSQL pour récupérer la liste des nœuds actifs.

2. **Récupération des nœuds** : Pour chaque nœud disposant de coordonnées GPS (latitude, longitude), le service extrait également l'altitude, la région et le modem utilisé.

3. **Calcul de couverture** : Le service fait appel au wrapper Python du projet Meshtastic Site Planner, qui lui-même exécute le binaire SPLAT!. Ce dernier télécharge les données d'élévation SRTM de la NASA et calcule la propagation du signal radio.

4. **Sauvegarde des résultats** : Le résultat de la simulation (fichier GeoTIFF) est converti en polygone GeoJSON, puis stocké dans la table `CoverageMaps` de la base de données.

5.2.3 Composants clés

Le service repose sur trois éléments principaux :

- **coverage_daemon.py** : Script Python principal qui orchestre la boucle et les appels aux différents composants.
- **Site Planner wrapper** (fichier `app/services/splat.py`) : Interface Python qui facilite l'appel au binaire SPLAT! et la gestion des données.
- **SPLAT! binary** (`splat/splat`) : Binaire C++ qui réalise les calculs de propagation radio à partir des données de terrain.

5.2.4 Code clé – Boucle principale

La boucle principale du démon exécute les étapes suivantes de façon continue :

```
while True:
    # 1. Connexion    la base de donn es
    conn = get_db_connection()

    # 2. R cup ration de tous les n uds avec coordonn es GPS
    nodes = get_active_nodes(conn)

    # 3. Pour chaque n ud , calcul de la couverture
    for node in nodes:
        if needs_recalculation(conn, node.id):
            # Appel    SPLAT! via le wrapper Site Planner
            geojson = calculate_node_coverage(node)

            # Sauvegarde dans la base de donn es
            save_coverage_map(conn, node.id, geojson)

    # 4. Attente avant la prochaine ex cution
    time.sleep(RUN_INTERVAL)
```

Listing 5.1: Boucle principale du service de couverture

5.2.5 Code clé – Appel à SPLAT!

L'initialisation de SPLAT! et l'exécution de la prédiction se font via le wrapper fourni par le projet Meshtastic Site Planner :

```
splat_service = Splat(splat_path="/opt/site-planner/splat")
request = CoveragePredictionRequest(
```

```
    lat=lat,            # Latitude du n ud
    lon=lon,            # Longitude du n ud
    height=height,      # Hauteur de l'antenne
    frequency=frequency, # Fr quence radio (MHz)
    tx_power=20,        # Puissance d' mission  (dBm)
    rx_sensitivity=-130 # Sensibilit  du r cepteur (dBm)
)
geotiff_data = splat_service.coverage_prediction(request)
```

Listing 5.2: Initialisation de SPLAT! et exécution de la prédiction

5.2.6 Stockage des résultats

Les résultats sont stockés dans la table `CoverageMaps` au format GeoJSON :

```
CREATE TABLE "CoverageMaps" (
    "NodeId" BIGINT PRIMARY KEY,
    "GeoJSON" JSONB NOT NULL,
    "CalculatedAt" TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    "Parameters" JSONB,
    FOREIGN KEY ("NodeId") REFERENCES "Nodes"("Id") ON DELETE CASCADE
);
```

Listing 5.3: Structure de la table CoverageMaps

5.2.7 Visualisation des logs

La figure ci-dessous montre les logs générés par le service lors de l'exécution des calculs SPLAT! pour plusieurs nœuds. On y observe le téléchargement des données de terrain, la conversion en format SDF, et la génération réussie des GeoTIFF.

```
zenlenx@SPLAT:~/Documents/meshnetwork/docker$ sudo docker compose logs --tail=20 --timestamps coverage
coverage-1  | 2026-03-23T21:18:10.663734997Z 2026-03-23 21:18:10,663 - app.services.splat - INFO - Initialized SPLAT! with terrain tile cache at '.splat_tiles' with a size limit of 1.0 GB.
coverage-1  | 2026-03-23T21:18:10.663761817Z 2026-03-23 21:18:10,663 - coverage-daemon - INFO - Running SPLAT! simulation for node 222221
coverage-1  | 2026-03-23T21:18:10.664113129Z 2026-03-23 21:18:10,663 - app.services.splat - INFO - Downloading N45E004.hgt.gz from elevation-tiles-prod/v2/skadi/N45/N45E004.hgt.gz...
coverage-1  | 2026-03-23T21:18:44.062627828Z 2026-03-23 21:18:44,062 - app.services.splat - INFO - Decompressing N45E004.hgt.gz into /tmp/tmpu963b9an/N45E004.hgt.
coverage-1  | 2026-03-23T21:18:44.236067498Z 2026-03-23 21:18:44,235 - app.services.splat - INFO - Downsampling /tmp/tmpu963b9an/N45E004.hgt to 3-arcsecond resolution.
coverage-1  | 2026-03-23T21:18:44.309522048Z 2026-03-23 21:18:44,309 - rasterio._env - WARNING - CPLE_AppDefined in The corner coordinates of the source are not properly aligned on plain l
atitude/longitude boundaries.
coverage-1  | 2026-03-23T21:18:44.314249270Z 2026-03-23 21:18:44,314 - app.services.splat - INFO - Successfully downsampled /tmp/tmpu963b9an/N45E004.hgt.
coverage-1  | 2026-03-23T21:18:44.314263407Z 2026-03-23 21:18:44,314 - app.services.splat - INFO - Converting /tmp/tmpu963b9an/N45E004.hgt to 45:46:355:356.sdf using /opt/site-planner/spla
t/srtm2sdf.
coverage-1  | 2026-03-23T21:18:45.684305098Z 2026-03-23 21:18:45,684 - app.services.splat - INFO - Successfully converted and cached 45:46:355:356.sdf.
coverage-1  | 2026-03-23T21:18:46.053509242Z 2026-03-23 21:18:46,053 - app.services.splat - INFO - Starting GeoTIFF generation from SPLAT! PPM and KML data.
coverage-1  | 2026-03-23T21:18:46.062700064Z 2026-03-23 21:18:46,062 - rasterio._env - INFO - GDAL signalled an error: err_no=1, msg='TIFFSetField:/vsimem/6f246f64-80d3-4390-bf5e-c7ed75a82
600/6f246f64-80d3-4390-bf5e-c7ed75a82600.tif: Cannot modify tag "PhotometricInterpretation" while writing'
coverage-1  | 2026-03-23T21:18:46.062804712Z 2026-03-23 21:18:46,062 - app.services.splat - INFO - GeoTIFF generation successful.
coverage-1  | 2026-03-23T21:18:46.062881795Z 2026-03-23 21:18:46,062 - app.services.splat - INFO - SPLAT! coverage prediction completed successfully.
coverage-1  | 2026-03-23T21:18:46.068172900Z 2026-03-23 21:18:46,068 - coverage-daemon - INFO - Successfully calculated coverage for node 222221
coverage-1  | 2026-03-23T21:18:46.078220828Z 2026-03-23 21:18:46,078 - coverage-daemon - INFO - Saved coverage map for node 222221
coverage-1  | 2026-03-23T21:18:51.079252590Z 2026-03-23 21:18:51,079 - coverage-daemon - ERROR - Failed to check recalculation for node 333331: name 'timezone' is not defined
coverage-1  | 2026-03-23T21:18:51.079282359Z 2026-03-23 21:18:51,079 - coverage-daemon - INFO - Calculating coverage for node 333331 - Marseille Centre
coverage-1  | 2026-03-23T21:18:51.085876340Z 2026-03-23 21:18:51,085 - app.services.splat - INFO - Initialized SPLAT! with terrain tile cache at '.splat_tiles' with a size limit of 1.0 GB.
coverage-1  | 2026-03-23T21:18:51.085897302Z 2026-03-23 21:18:51,085 - coverage-daemon - INFO - Running SPLAT! simulation for node 333331
coverage-1  | 2026-03-23T21:18:51.086238115Z 2026-03-23 21:18:51,086 - app.services.splat - INFO - Downloading N43E005.hgt.gz from elevation-tiles-prod/v2/skadi/N43/N43E005.hgt.gz...
```

Figure 5.2: Logs du service de couverture pendant les calculs SPLAT!

5.3 Service de Redondance

5.3.1 Architecture du service

Le service de redondance analyse les zones de couverture générées par le service précédent pour identifier les chevauchements. Comme illustré sur la figure ci-dessous, il fonctionne en microservice conteneurisé qui communique avec la base de données PostgreSQL.

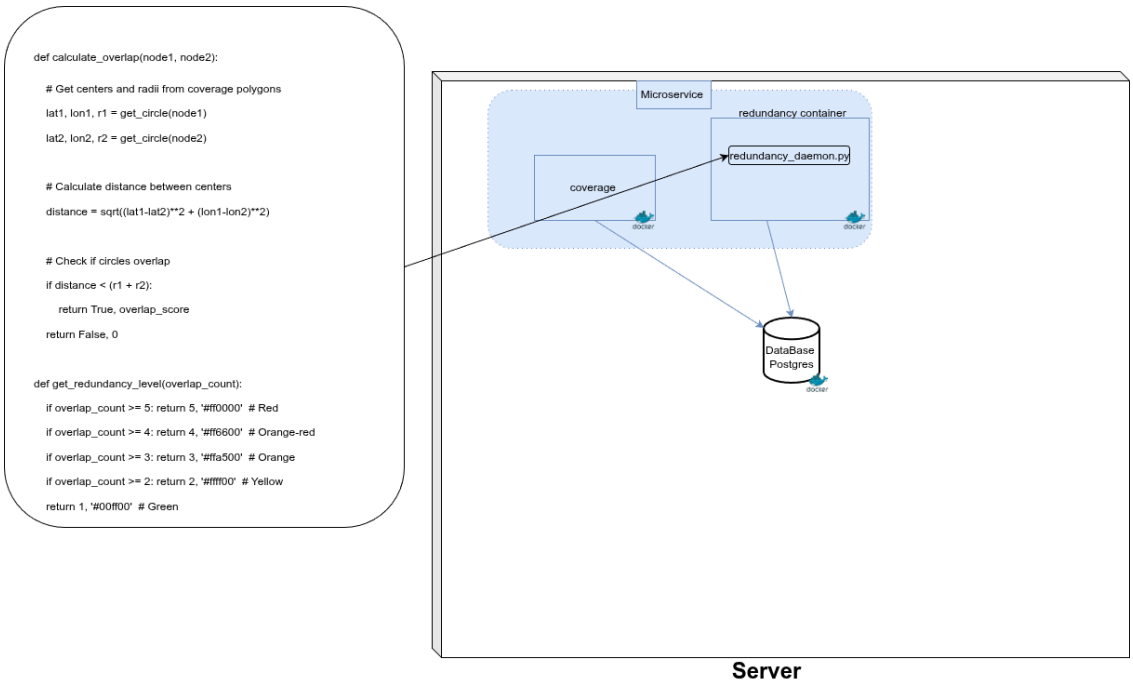

Figure 5.3: Architecture et flux d'exécution du service de redondance

5.3.2 Objectif et principe

Le service de redondance répond à une question clé : *« Combien de nœuds couvrent la même zone géographique ? »*

Plus une zone est couverte par plusieurs nœuds, plus le réseau est résilient. Le service analyse donc les polygones de couverture stockés dans `CoverageMaps` pour :

- Calculer les chevauchements entre les zones de couverture
- Attribuer un **niveau de redondance** (1 à 5) à chaque nœud
- Associer une **couleur** (vert → rouge) pour la visualisation
- Identifier les **clusters** de nœuds redondants

5.3.3 Algorithme de calcul des chevauchements

Pour chaque paire de nœuds, le service extrait les centres et les rayons de leurs polygones de couverture (approximation circulaire). Il calcule ensuite la distance entre les centres :

```
def calculate_overlap(node1, node2):
    # Extraction des centres et rayons
    lat1, lon1, r1 = get_circle(node1)
    lat2, lon2, r2 = get_circle(node2)

    # Calcul de la distance entre les centres
    distance = sqrt((lat1-lat2)**2 + (lon1-lon2)**2)

    # V rification du chevauchement
    if distance < (r1 + r2):
        return True, overlap_score
    return False, 0
```

Listing 5.4: Fonction de calcul de chevauchement

5.3.4 Niveaux de redondance et couleurs

Le service attribue un niveau de redondance en fonction du nombre de chevauchements :

```
def get_redundancy_level(overlap_count):
    if overlap_count >= 5: return 5, '#ff0000'   # Rouge - tr s redondant
    if overlap_count >= 4: return 4, '#ff6600'   # Orange-rouge - hautement
   redondant
    if overlap_count >= 3: return 3, '#ffa500'   # Orange - bonne redondance
    if overlap_count >= 2: return 2, '#ffff00'   # Jaune - redondance moyenne
    return 1, '#00ff00'                          # Vert - faible redondance
```

Listing 5.5: Attribution des niveaux et couleurs

Le tableau 5.1 résume la correspondance entre le nombre de chevauchements et la couleur associée .

Niveau	Couleur	Signification
1	Vert	1 nœud (aucun chevauchement)
2	Jaune	2 nœuds se chevauchent
3	Orange	3 nœuds se chevauchent
4	Orange-rouge	4 nœuds se chevauchent
5	Rouge	5+ nœuds se chevauchent

Table 5.1: Niveaux de redondance

5.3.5 Stockage des résultats

Deux tables sont créées pour stocker les résultats :

- `Redundancy` : métriques par nœud (nombre de chevauchements, niveau, couleur)
- `RedundancyMap` : agrégation des zones par niveau (clusters)

```sql
CREATE TABLE "Redundancy" (
    "NodeId" BIGINT PRIMARY KEY,
    "Data" JSONB NOT NULL,
    "CalculatedAt" TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    FOREIGN KEY ("NodeId") REFERENCES "Nodes"("Id") ON DELETE CASCADE
);

CREATE TABLE "RedundancyMap" (
    "Id" SERIAL PRIMARY KEY,
    "Level" INTEGER NOT NULL,
    "Color" VARCHAR(10) NOT NULL,
    "NodeIds" JSONB NOT NULL,
    "CenterLat" DOUBLE PRECISION,
    "CenterLon" DOUBLE PRECISION,
    "CalculatedAt" TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);
```

Listing 5.6: Création des tables de redondance

5.3.6 Composants du service

Le service de redondance repose sur deux éléments principaux :

- **redundancy_daemon.py** : Script Python qui orchestre le calcul des chevauchements.
- **psycopg2-binary** : Bibliothèque Python pour l'interaction avec PostgreSQL.

5.3.7 Dockerfile du service

Le conteneur est minimal, se contentant d'exécuter le script Python :

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir psycopg2-binary

COPY redundancy_daemon.py .

CMD ["python", "-u", "redundancy_daemon.py"]
```

Listing 5.7: Dockerfile du service de redondance

5.3.8 Visualisation des logs

La figure ci-dessous montre les logs générés par le service de redondance. On y observe l'analyse périodique des 8 cartes de couverture, avec l'agrégation des nœuds par niveau (3 nœuds en niveau 3 orange, 2 en niveau 2 jaune, 3 en niveau 1 vert).

```
zenlenx@SPLAT:~/Documents/meshnetwork/docker$ sudo docker compose logs --tail=20 --timestamps redundancy
redundancy-1  | 2026-03-23T21:17:28.233213693Z 2026-03-23 21:17:28,232 - redundancy-daemon - INFO -   Level 3 (#ffa500): 3 nodes
redundancy-1  | 2026-03-23T21:17:28.233215256Z 2026-03-23 21:17:28,233 - redundancy-daemon - INFO - Cycle complete. Sleeping for 60 seconds
redundancy-1  | 2026-03-23T21:18:28.245077954Z 2026-03-23 21:18:28,244 - redundancy-daemon - INFO - Found 8 coverage maps
redundancy-1  | 2026-03-23T21:18:28.253265968Z 2026-03-23 21:18:28,252 - redundancy-daemon - INFO - Redundancy calculated for 8 nodes
redundancy-1  | 2026-03-23T21:18:28.253282643Z 2026-03-23 21:18:28,253 - redundancy-daemon - INFO -   Level 1 (#00ff00): 3 nodes
redundancy-1  | 2026-03-23T21:18:28.253288826Z 2026-03-23 21:18:28,253 - redundancy-daemon - INFO -   Level 2 (#ffff00): 2 nodes
redundancy-1  | 2026-03-23T21:18:28.253293837Z 2026-03-23 21:18:28,253 - redundancy-daemon - INFO -   Level 3 (#ffa500): 3 nodes
redundancy-1  | 2026-03-23T21:18:28.253298616Z 2026-03-23 21:18:28,253 - redundancy-daemon - INFO - Cycle complete. Sleeping for 60 seconds
redundancy-1  | 2026-03-23T21:19:28.271193401Z 2026-03-23 21:19:28,270 - redundancy-daemon - INFO - Found 8 coverage maps
redundancy-1  | 2026-03-23T21:19:28.295052279Z 2026-03-23 21:19:28,292 - redundancy-daemon - INFO - Redundancy calculated for 8 nodes
redundancy-1  | 2026-03-23T21:19:28.295075684Z 2026-03-23 21:19:28,292 - redundancy-daemon - INFO -   Level 1 (#00ff00): 3 nodes
redundancy-1  | 2026-03-23T21:19:28.295082167Z 2026-03-23 21:19:28,292 - redundancy-daemon - INFO -   Level 2 (#ffff00): 2 nodes
redundancy-1  | 2026-03-23T21:19:28.295120070Z 2026-03-23 21:19:28,292 - redundancy-daemon - INFO -   Level 3 (#ffa500): 3 nodes
redundancy-1  | 2026-03-23T21:19:28.295126432Z 2026-03-23 21:19:28,292 - redundancy-daemon - INFO - Cycle complete. Sleeping for 60 seconds
redundancy-1  | 2026-03-23T21:20:28.304263669Z 2026-03-23 21:20:28,303 - redundancy-daemon - INFO - Found 8 coverage maps
redundancy-1  | 2026-03-23T21:20:28.323553326Z 2026-03-23 21:20:28,321 - redundancy-daemon - INFO - Redundancy calculated for 8 nodes
redundancy-1  | 2026-03-23T21:20:28.323572682Z 2026-03-23 21:20:28,321 - redundancy-daemon - INFO -   Level 1 (#00ff00): 3 nodes
redundancy-1  | 2026-03-23T21:20:28.323577542Z 2026-03-23 21:20:28,321 - redundancy-daemon - INFO -   Level 2 (#ffff00): 2 nodes
redundancy-1  | 2026-03-23T21:20:28.323581500Z 2026-03-23 21:20:28,321 - redundancy-daemon - INFO -   Level 3 (#ffa500): 3 nodes
redundancy-1  | 2026-03-23T21:20:28.323585337Z 2026-03-23 21:20:28,321 - redundancy-daemon - INFO - Cycle complete. Sleeping for 60 seconds
```

Figure 5.4: Logs du service de redondance avec l'agrégation par niveaux

5.4 Intégration d'une Architecture Micro Frontend

5.4.1 Contexte et Motivation

Dans le cadre de l'évolution de l'application **Meshtastic MQTT Explorer**, une nouvelle fonctionnalité a été développée : la **carte de couverture radio** des nœuds du réseau LoRa. Cette fonctionnalité permet de visualiser la portée estimée de chaque nœud sur une carte interactive.

L'application existante est développée en **Blazor Server** (.NET 9), une technologie C# côté serveur. Ajouter la carte de couverture directement dans ce projet aurait nécessité de modifier en profondeur le code source existant, ce qui présentait plusieurs risques :

- Introduire des bugs dans des fonctionnalités déjà stables
- Augmenter la complexité du projet principal
- Rendre difficile la maintenance indépendante de cette fonctionnalité

Pour éviter ces problèmes, il a été décidé d'adopter une approche **Micro Frontend (MFE)**.

5.4.2 Concept du Micro Frontend

L'architecture Micro Frontend est une approche qui consiste à **décomposer une application web en plusieurs sous-applications indépendantes**, chacune développée, déployée et maintenue séparément.

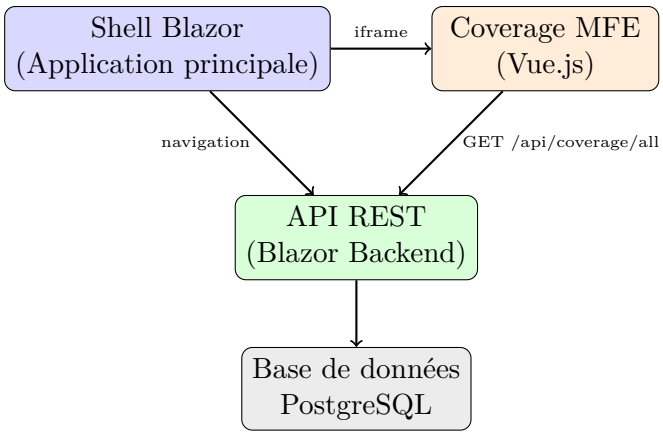

Figure 5.5: Architecture Micro Frontend : le shell Blazor intègre le MFE Vue.js via une iframe

Dans notre cas, l'application Blazor joue le rôle de **shell** (conteneur principal), tandis que la carte de couverture est encapsulée dans un **micro frontend indépendant** développé en **Vue.js 3**.

5.4.3 Choix Technologiques

5.4.3.1 Vue.js 3 pour le Micro Frontend

Le micro frontend de couverture a été développé avec **Vue.js 3**, pour les raisons suivantes :

- **Légèreté** : Vue.js produit des bundles de taille réduite, ce qui minimise le temps de chargement de la page
- **Compatibilité Leaflet** : la bibliothèque cartographique Leaflet utilisée dans le projet existant s'intègre naturellement dans Vue.js
- **Indépendance technologique** : l'utilisation de Vue.js démontre qu'un MFE peut être développé avec n'importe quelle technologie, indépendamment du shell

5.4.3.2 Intégration via iframe

Le mécanisme d'intégration choisi est l'**iframe HTML**, qui offre une isolation totale entre les deux applications :

```
@page "/coverage"

<PageTitle>Carte de couverture</PageTitle>

<iframe
    src="http://localhost:3001"
    style="width: 100%; height: calc(100vh - 130px);
           border: none; display: block;"
    title="Carte de couverture"
    loading="lazy" />
```

Listing 5.8: CoverageMapPage.razor : le shell Blazor délègue l'affichage au MFE

La page Blazor `/coverage` est réduite à **3 lignes** : elle ne contient plus aucune logique métier, uniquement un conteneur qui charge le MFE.

5.4.4 Architecture de Déploiement

Chaque composant est déployé dans son propre **container Docker** :

```
services:
  front:            # Shell Blazor         - port 80
```

```
  coverage-mfe:     # Micro Frontend Vue  - port 3001
```

Listing 5.9: Extrait du docker-compose.yml

Le micro frontend expose son interface sur le **port 3001** et communique avec l'API Blazor via un **proxy Nginx** intégré, qui redirige les requêtes `/api/coverage/*` et `/api/coverage/*` vers le backend :

```
resolver 127.0.0.11 valid=30s;

location /api/ {
    set $backend http://front:8080;
    proxy_pass $backend;
}
```

Listing 5.10: nginx.conf du MFE : proxy vers l'API Blazor

5.4.5 Résultats

L'intégration du micro frontend a produit les résultats suivants, illustrés dans les captures d'écran ci-dessous.

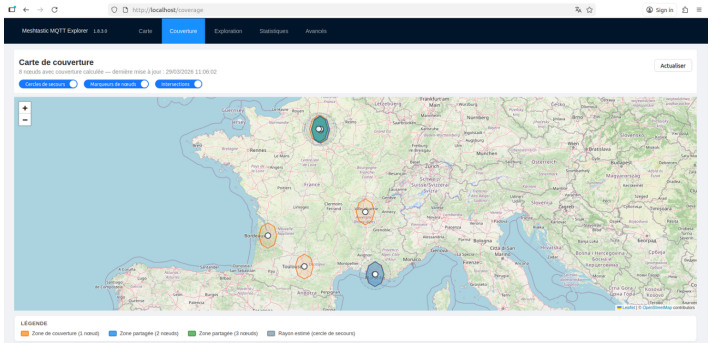

Figure 5.6: Page `/coverage` : carte de couverture affichant 5 nœuds avec leurs zones de portée. Les polygones orange représentent la couverture calculée, les intersections representées par la couleur bleu et les cercles bleus pointillés sont les cercles de secours

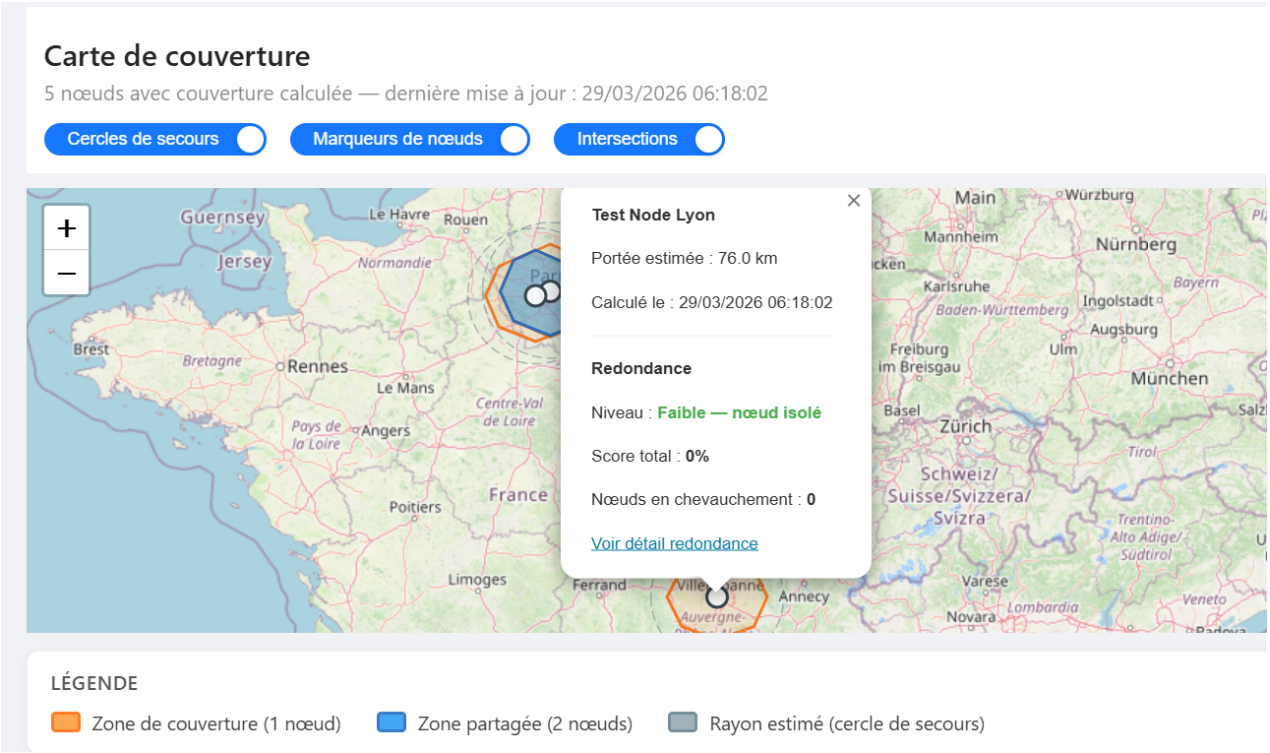

Figure 5.7: Page `/coverage` : affichage des informations des redondance pour un noeud.

Les points clés du résultat obtenu sont :

- **Isolation complète** : le MFE Vue.js fonctionne indépendamment du shell Blazor
- **Modifications minimales** au code Blazor existant : seulement 3 lignes modifiées dans `CoverageMapPage.r`
- **Design identique** au reste de l'application : le header, les toggles et la carte reproduisent fidèlement le style Ant Design du projet original
- **Déploiement indépendant** : le MFE peut être mis à jour sans rebuilder ni redémarrer le shell Blazor
- **Superposition des zones colorées** : les nœuds proches (Paris et Versailles) montrent un chevauchement partiel de leurs zones de couverture, représenté en bleu, validant la pertinence de la visualisation des intersections
- **Données de redondance par nœud** : un clic sur un marqueur affiche le niveau de redondance (isolé, partagé, suffisant), le score de chevauchement en pourcentage, et la liste des nœuds voisins avec leur score d'overlap individuel, consommés depuis l'endpoint `GET /api/redundancy/all`
- **Légende dynamique** : la légende sous la carte s'adapte automatiquement aux données présentes — elle n'affiche que les niveaux d'intersection effectivement calculés (2 nœuds, 3 nœuds, etc.) et

reflète l'état des toggles actifs

5.5 Conclusion

Cette expérience démontre qu'il est possible d'enrichir une application existante avec de nouvelles fonctionnalités **sans modifier son code source**, en s'appuyant sur une architecture micro frontend. L'approche par iframe, bien que simple, offre une isolation totale et une grande flexibilité technologique. Elle constitue une base solide pour ajouter d'autres fonctionnalités indépendantes à l'avenir, comme l'intégration du simulateur de couverture SPLAT! développé dans le projet `meshtastic-site-planner`.

Chapitre 6

Conclusion Générale

La réalisation de ce projet a permis de transformer une interface de monitoring classique en un outil d'aide à la planification réseau performant. Grâce à l'utilisation de Docker, nous avons réussi à orchestrer des technologies variées (Python, Vue.js, PostgreSQL) de manière totalement transparente et portable.Le bilan technique est particulièrement positif : l'approche Micro Frontend a prouvé son efficacité en permettant d'ajouter des fonctionnalités complexes avec seulement trois lignes de modification dans le projet initial. Au-delà de l'aspect technique, ce travail s'inscrit dans une démarche Open Source, offrant des services modulaires et réutilisables qui pourraient bénéficier à l'ensemble de la communauté Meshtastic. Ce projet constitue ainsi une base solide pour de futures extensions, démontrant qu'une architecture découplée est la clé de la maintenabilité et de l'évolutivité des systèmes logiciels modernes.

Bibliography

[1] Meshtastic Project Contributors. *Documentation officielle de Meshtastic*, https://meshtastic.org/docs. Consulté en mars 2026.

[2] John A. Magliacane, KD2BD. *SPLAT! - A Terrain Analysis Program for Amateur Radio*, https://www.qsl.net/kd2bd/splat.html. Version 1.4.2.

[3] NASA JPL. *Shuttle Radar Topography Mission (SRTM)*, https://www2.jpl.nasa.gov/srtm. Données d'élévation globales.

[4] MQTT Community. *Protocole MQTT - Documentation officielle*, https://mqtt.org.

[5] PostgreSQL Global Development Group. *PostgreSQL Documentation*, https://www.postgresql.org/docs.

[6] Docker Inc. *Docker Documentation*, https://docs.docker.com.

[7] Vue.js Core Team. *Vue.js Guide*, https://vuejs.org/guide.

[8] Volodymyr Agafonkin. *Leaflet - an open-source JavaScript library for interactive maps*, https://leafletjs.com.

[9] Valentintintin. *Meshtastic MQTT Explorer*, https://github.com/valentintintin/meshtastic-mqtt-explorer. Dépôt GitHub.

[10] Meshtastic Contributors. *Meshtastic Site Planner*, https://github.com/meshtastic/meshtastic-site-planner. Dépôt GitHub.